# Data Science & ML: Exam Answers (Q1-Q36)

Serial-wise explanations with examples and code (Python, Pandas, NumPy, scikit-learn).

All code assumes these imports:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

## Q1. Difference between ML, DS and AI (4 points)

**Artificial Intelligence (AI):** the broad goal of making machines behave intelligently (reasoning, understanding language, seeing, deciding). **Machine Learning (ML):** a subset of AI where machines learn patterns from data instead of being explicitly programmed. **Data Science (DS):** an interdisciplinary field that uses statistics, programming and domain knowledge to extract insights from data. It uses ML as one of its tools.

| Point | AI | ML | DS |
|---|---|---|---|
| Scope | Broadest: simulate human intelligence | Subset of AI: learn from data | Overlaps both: insights from data |
| Goal | Build intelligent systems | Build models that predict/classify | Find patterns and support decisions |
| Output | Intelligent action/behaviour | Trained model/predictions | Reports, dashboards, models |
| Techniques | Rules, search, ML, deep learning | Regression, trees, KNN, neural nets | Statistics, EDA, visualization, ML |
| Example | Self-driving car, Siri | Spam filter, price prediction | Analysing sales data to plan stock |

Remember it as: **DS** = analyse data, **ML** = learn from data, **AI** = act intelligently.

# Q2. Features of ML and applications of ML (6 points)

### Features of ML

1. **Learns from data:** improves with experience, no hard-coded rules.
2. **Automation:** once trained, it makes predictions automatically.
3. **Handles large, multi-dimensional data:** finds patterns humans miss.
4. **Continuous improvement:** accuracy rises as more data arrives.
5. **Adaptability:** can be retrained when data changes.
6. **Wide applicability:** works across domains (finance, health, retail).

### Applications of ML

1. Spam email detection (Gmail)
2. Recommendation systems (Netflix, Amazon)
3. Fraud detection in banking
4. Medical diagnosis (disease prediction from scans)
5. Self-driving cars
6. Speech recognition and chatbots (Alexa, Siri)
7. House price / stock prediction

# Q3. Difference between sample(), head() and tail() with example

- **head(n):** first n rows (default 5).
- **tail(n):** last n rows (default 5).
- **sample(n):** n randomly chosen rows from anywhere. Use `random_state` for reproducibility.

```
df = pd.DataFrame({
    'Name': ['A','B','C','D','E','F','G','H'],
    'Marks': [80, 65, 90, 70, 55, 85, 60, 95]
})

print(df.head(3))      # rows 0,1,2
print(df.tail(3))      # rows 5,6,7
print(df.sample(3, random_state=42))   # 3 random rows
```

| Method | Rows returned | Order |
|---|---|---|
| head() | First n | Fixed (top) |
| tail() | Last n | Fixed (bottom) |
| sample() | Random n | Random, changes each run |

Use `head()` for a quick peek, `tail()` to check the end (e.g. time series latest data), `sample()` to get an unbiased glance at the whole data.

## Q4. Difference between info() and describe()

**info()** gives the structure of the DataFrame: number of rows, columns, column names, data types, non-null counts and memory usage. It is used to spot missing values and wrong dtypes.

**describe()** gives statistical summary of numeric columns: count, mean, std, min, 25%, 50%, 75%, max. Use `include='all'` or `include='object'` for text columns.

```
df = pd.DataFrame({
    'Age': [25, 30, None, 40],
    'City': ['Delhi','Pune','Delhi','Agra']
})

df.info()
print(df.describe())
print(df.describe(include='object'))   # count, unique, top, freq
```

| info() | describe() |
|---|---|
| Structure/metadata | Statistics |
| Shows dtypes, non-null counts, memory | Shows mean, std, quartiles, min, max |
| Prints directly, returns None | Returns a DataFrame |

## Q5. Dependent and independent variable

- **Independent variable (feature / input / X):** the variable used to make a prediction. It is not affected by other variables.
- **Dependent variable (target / output / label / y):** the variable we want to predict. Its value depends on the independent variables.

**Example:** Predicting house price.

| Area (sqft) | Bedrooms | Price |
|---|---|---|
| 1000 | 2 | 50 lakh |
| 1500 | 3 | 75 lakh |

Independent: Area, Bedrooms. Dependent: Price.

```
X = df[['Area', 'Bedrooms']]   # independent
y = df['Price']                # dependent
```

## Q6. How many dependent and independent variables can a dataset have?

- **Independent variables:** one or many (any number of features). One feature = simple model, many = multiple/multivariate model.
- **Dependent variable:** normally **exactly one** (single target) in standard regression/classification. In special cases (multi-output / multi-label problems) there can be more than one.

```
# one target, many features
X = df.drop('Price', axis=1)   # all other columns
y = df['Price']                # one target
```

Rule: *n columns = (n-1) independent + 1 dependent* in a typical supervised dataset. In unsupervised learning (e.g. clustering) there is **no** dependent variable.

## Q7. How to remove duplicates from a dataset

Use `duplicated()` to find them and `drop_duplicates()` to remove them.

```
df = pd.DataFrame({
    'Name': ['A','B','A','C','B'],
    'Age':  [20, 21, 20, 22, 25]
})

print(df.duplicated())          # True for the repeated row (index 2)
print(df.duplicated().sum())    # count of duplicates

clean = df.drop_duplicates()                        # keep first occurrence
clean2 = df.drop_duplicates(subset=['Name'])        # based on one column
clean3 = df.drop_duplicates(keep='last')            # keep last one
clean4 = df.drop_duplicates(keep=False)             # remove all duplicated
rows
df.drop_duplicates(inplace=True, ignore_index=True)  # change original, reset
index
```

`keep` can be `'first'` (default), `'last'` or `False`.

## Q8. Use of drop() function with example (explain parameters)

`drop()` removes rows or columns from a DataFrame.

**Syntax:** `df.drop(labels=None, axis=0, index=None, columns=None, level=None, inplace=False, errors='raise')`

| Parameter | Meaning |
|---|---|
| labels | Name(s) of row index or column to drop |
| axis | 0 = rows (default), 1 = columns |
| index | Row labels directly (alternative to labels + axis=0) |
| columns | Column names directly (alternative to axis=1) |
| inplace | False = returns new df; True = modifies original |
| errors | 'raise' (error if label missing) or 'ignore' |

```
df = pd.DataFrame({
    'ID': [1,2,3],
    'Name': ['A','B','C'],
    'Age': [20,21,22]
})

df.drop('Age', axis=1)               # drop column
df.drop(columns=['Name','Age'])      # drop multiple columns
df.drop(0, axis=0)                   # drop row with index 0
df.drop(index=[0,2])                 # drop multiple rows
df.drop('Salary', axis=1, errors='ignore')   # no error even if absent
```

## Q9. Use of inplace parameter in drop()

`inplace` decides whether the change is made on the **original** DataFrame or on a **copy**.

- `inplace=False` (default): original unchanged; a new DataFrame is returned.
- `inplace=True`: original is modified permanently; returns `None`.

```
df = pd.DataFrame({'A':[1,2], 'B':[3,4]})

new_df = df.drop('B', axis=1)        # df still has A and B
print(df.columns)                    # ['A', 'B']

df.drop('B', axis=1, inplace=True)   # df itself changed
print(df.columns)                    # ['A']

x = df.drop('A', axis=1, inplace=True)
print(x)                             # None  (common mistake: don't assign)
```

## Q10. What are activation functions?

An activation function is applied to the output of a neuron in a neural network to decide whether it should "fire" and to add **non-linearity**. Without it, a network of many layers would behave like a single linear model and could not learn complex patterns.

| Function | Formula | Range | Use |
|---|---|---|---|
| Sigmoid | 1/(1+e^-x) | 0 to 1 | Binary classification output |

| Function | Formula | Range | Use |
|---|---|---|---|
| Tanh | (e^x - e^-x)/(e^x + e^-x) | -1 to 1 | Hidden layers (zero-centred) |
| ReLU | max(0, x) | 0 to infinity | Most popular for hidden layers |
| Leaky ReLU | max(0.01x, x) | -inf to inf | Avoids "dead neurons" |
| Softmax | e^xi / sum(e^xj) | 0 to 1 (sums to 1) | Multi-class output layer |

```
def sigmoid(x): return 1 / (1 + np.exp(-x))
def relu(x):    return np.maximum(0, x)

x = np.array([-2, -1, 0, 1, 2])
print(sigmoid(x))   # [0.119 0.269 0.5 0.731 0.881]
print(np.tanh(x))
print(relu(x))      # [0 0 0 1 2]
```

## Q11. Explain dropna() function

`dropna()` removes rows or columns that contain missing values (NaN).

**Syntax:** `df.dropna(axis=0, how='any', thresh=None, subset=None, inplace=False)`

| Parameter | Meaning |
|---|---|
| axis | 0 = drop rows (default), 1 = drop columns |
| how | 'any' = drop if any NaN; 'all' = drop only if all NaN |
| thresh | Keep row/column only if at least this many non-null values |
| subset | Check NaN only in these columns |
| inplace | Modify original if True |

```
df = pd.DataFrame({
    'A': [1, np.nan, 3, np.nan],
    'B': [5, 6, np.nan, np.nan],
    'C': [7, 8, 9, np.nan]
})

df.dropna()                    # only row 0 survives (no NaN)
df.dropna(how='all')           # drops only row 3 (all NaN)
df.dropna(axis=1)              # drops columns with any NaN (all three here)
df.dropna(thresh=2)            # keep rows with >= 2 valid values
df.dropna(subset=['A'])        # drop rows where A is NaN
```

Caution: dropping too many rows loses data. Prefer `fillna()` when missing values are many.

## Q12. Use of isnull() with example

`isnull()` (alias `isna()`) returns a boolean DataFrame/Series: **True** where a value is missing (NaN/None), **False** otherwise. `notnull()` is its opposite.

```
df = pd.DataFrame({
    'Name': ['A','B','C'],
    'Age':  [25, np.nan, 30],
    'City': ['Delhi', 'Pune', None]
})

print(df.isnull())          # True/False grid
print(df.isnull().sum())    # missing count per column -> Age 1, City 1
print(df.isnull().sum().sum())   # total missing = 2
print(df[df['Age'].isnull()])    # rows where Age is missing

# fix: fill missing values
df['Age'] = df['Age'].fillna(df['Age'].mean())
```

It is the first step of data cleaning: find missing values with `isnull().sum()`, then handle them with `dropna()` or `fillna()`.

# Q13. Explain any 3 data visualization libraries and any 4 functions of any library

**1. Matplotlib:** the base plotting library in Python. Gives full control over line, bar, scatter, histogram and pie charts.

**2. Seaborn:** built on Matplotlib. Gives attractive statistical plots (heatmap, boxplot, pairplot) with less code and works directly with DataFrames.

**3. Plotly:** creates interactive charts (zoom, hover, pan) for dashboards and web apps. Others: Bokeh, Altair.

**4 functions of Matplotlib (pyplot):**

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]

# 1. plot(): line chart
plt.plot(x, y, color='blue', marker='o')
plt.title('Line Plot'); plt.xlabel('X'); plt.ylabel('Y')
plt.show()

# 2. bar(): bar chart
plt.bar(['A','B','C'], [5, 7, 3], color='green')
plt.show()

# 3. hist(): histogram (distribution of one variable)
plt.hist(np.random.randn(500), bins=20)
plt.show()

# 4. scatter(): relationship between two variables
plt.scatter(x, y, color='red')
plt.show()
```

Other useful functions: `pie()`, `boxplot()`, `subplot()`, `legend()`, `savefig()`.

# Q14. Difference between DataFrame and Series

- **Series:** one-dimensional labelled array. It holds a single column of data of one dtype.

- **DataFrame:** two-dimensional labelled table (rows and columns). It is a collection of Series sharing the same index.

```
s = pd.Series([10, 20, 30], index=['a','b','c'], name='Marks')
print(s)

df = pd.DataFrame({
    'Name': ['A','B','C'],
    'Marks': [10, 20, 30]
})
print(type(df['Marks']))   # pandas Series: one column of a DataFrame
print(type(df[['Marks']])) # DataFrame
```

| Series | DataFrame |
|---|---|
| 1-D | 2-D |
| Single column | Multiple columns |
| Same dtype | Different dtypes per column |
| Created by `pd.Series()` | Created by `pd.DataFrame()` |
| Like a column in Excel | Like a whole Excel sheet |

## Q15. What is ML, types of ML

**Machine Learning** is the science of making computers learn patterns from data and make predictions or decisions without being explicitly programmed (Arthur Samuel's idea).

**Types of ML**

1. **Supervised learning:** trained on **labelled** data (input + known output). Two kinds:
   - Regression: predict a number (house price). Algorithms: Linear Regression, Random Forest Regressor.
   - Classification: predict a category (spam / not spam). Algorithms: Logistic Regression, KNN, Decision Tree, SVM.
2. **Unsupervised learning: no labels**; the model finds hidden structure. Examples: clustering (K-Means, customer segmentation), dimensionality reduction (PCA), association rules.

3. **Semi-supervised learning:** small labelled data + large unlabelled data (e.g. photo tagging).
4. **Reinforcement learning:** an agent learns by trial and error using rewards and penalties (game playing, robotics, self-driving).

```
from sklearn.linear_model import LinearRegression   # supervised
from sklearn.cluster import KMeans                  # unsupervised

X = [[1],[2],[3],[4]]; y = [2,4,6,8]
print(LinearRegression().fit(X, y).predict([[5]]))   # [10.]
print(KMeans(n_clusters=2, n_init=10).fit_predict(X))
```

## Q16. Explain flatten() and ravel() function

Both convert a multi-dimensional NumPy array into a **1-D array**.

- **flatten():** always returns a **copy**. Changing the result does not affect the original.
- **ravel():** returns a **view** when possible (no new memory), so changing the result can change the original. Faster and more memory efficient.

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

f = a.flatten()
r = a.ravel()
print(f)   # [1 2 3 4 5 6]
print(r)   # [1 2 3 4 5 6]

f[0] = 100
print(a[0,0])   # 1   -> original unchanged (copy)

r[0] = 100
print(a[0,0])   # 100 -> original changed (view)
```

Both accept `order='C'` (row-wise, default) or `'F'` (column-wise).

## Q17. How to convert an array into a DataFrame

Pass the NumPy array to `pd.DataFrame()` and optionally give `columns` and `index`.

```
arr = np.array([[1, 'A', 80],
                [2, 'B', 90]], dtype=object)

df = pd.DataFrame(arr, columns=['ID', 'Name', 'Marks'])
print(df)

# numeric 2-D array
arr2 = np.array([[10, 20], [30, 40], [50, 60]])
df2 = pd.DataFrame(arr2, columns=['X', 'Y'], index=['r1', 'r2', 'r3'])
print(df2)

# 1-D array -> Series, or one-column DataFrame
arr1 = np.array([1, 2, 3])
print(pd.Series(arr1))
print(pd.DataFrame(arr1, columns=['Value']))
```

If you do not give columns, pandas uses 0, 1, 2... as column names.

## Q18. Difference between type() and dtype

- **type():** a Python built-in function that tells the **class of the object** itself.
- **dtype:** an attribute of NumPy arrays / pandas Series that tells the **data type of the elements** stored inside.

```
a = np.array([1, 2, 3])
print(type(a))    # <class 'numpy.ndarray'>   -> the object is an ndarray
print(a.dtype)    # int64                      -> elements are integers

s = pd.Series([1.5, 2.5])
print(type(s))    # <class 'pandas.core.series.Series'>
print(s.dtype)    # float64

df = pd.DataFrame({'A':[1,2], 'B':['x','y']})
print(df.dtypes)  # A int64, B object  (note: dtypes for a DataFrame)
```

Change dtype with `astype()`: `a.astype('float32')`.

# Q19. What is encoding? Explain any 2 encoding techniques

**Encoding** converts categorical (text) data into numbers, because ML algorithms work only with numeric input.

**1. Label Encoding:** assigns each category an integer (0, 1, 2...). Best for **ordinal** data (has order) like Low < Medium < High. Problem: for nominal data it creates a false order.

**2. One-Hot Encoding:** creates a **separate binary column** for each category (1 if present, else 0). Best for **nominal** data (no order) like colour or city. Problem: increases columns when there are many categories.

```
from sklearn.preprocessing import LabelEncoder

df = pd.DataFrame({'Color': ['Red','Green','Blue','Green']})

# Label encoding
df['Color_label'] = LabelEncoder().fit_transform(df['Color'])
# Blue=0, Green=1, Red=2

# One-hot encoding
ohe = pd.get_dummies(df['Color'], prefix='Color', dtype=int)
print(ohe)
#    Color_Blue  Color_Green  Color_Red
# 0           0            0          1
# 1           0            1          0
# 2           1            0          0
# 3           0            1          0
```

Other techniques: Ordinal Encoding, Target Encoding, Binary Encoding, Frequency Encoding.

# Q20. Explain any 4 functions of NumPy

```
# 1. np.array(): create an array
a = np.array([1, 2, 3, 4])

# 2. np.arange(start, stop, step): range of values
print(np.arange(0, 10, 2))      # [0 2 4 6 8]

# 3. np.zeros() / np.ones(): arrays filled with 0 / 1
print(np.zeros((2, 3)))

# 4. np.linspace(start, stop, num): evenly spaced numbers
print(np.linspace(0, 1, 5))     # [0.  0.25 0.5  0.75 1. ]
```

More: `np.mean()`, `np.sum()`, `np.max()`, `np.sqrt()`, `np.dot()`, `np.random.rand()`, `np.reshape()`, `np.unique()`.

```
b = np.array([10, 20, 30, 40])
print(np.mean(b), np.sum(b), np.max(b), np.std(b))
```

# Q21. Use of reshape() function with example

`reshape()` changes the **shape** (rows x columns) of an array without changing its data. The total number of elements must stay the same. Use `-1` to let NumPy compute one dimension automatically.

```
a = np.arange(1, 7)          # [1 2 3 4 5 6], shape (6,)

print(a.reshape(2, 3))
# [[1 2 3]
#  [4 5 6]]

print(a.reshape(3, 2))
# [[1 2]
#  [3 4]
#  [5 6]]

print(a.reshape(-1, 2).shape)   # (3, 2)  -1 means 'figure it out'
print(a.reshape(1, -1).shape)   # (1, 6)  commonly used in scikit-learn

# a.reshape(4, 2)  -> ValueError (6 elements cannot fit 8 cells)
```

Common use: `X.reshape(-1, 1)` to convert a 1-D feature into the 2-D format that scikit-learn needs.

## Q22. Difference between shallow copy and deep copy

- **Shallow copy:** creates a new outer object but the **inner (nested) objects are shared** with the original. Changing a nested item affects both.
- **Deep copy:** creates a completely **independent** copy including all nested objects. No effect on the original.

```
import copy

original = [[1, 2], [3, 4]]

shallow = copy.copy(original)
deep = copy.deepcopy(original)

original[0][0] = 99
print(shallow)   # [[99, 2], [3, 4]]  -> changed (shared inner list)
print(deep)      # [[1, 2], [3, 4]]   -> unchanged

# In pandas / NumPy
df2 = df.copy()              # deep=True by default (independent)
df3 = df.copy(deep=False)    # shallow
b = a.copy()                 # NumPy copy is independent
c = a.view()                 # view shares data
```

| Shallow | Deep |
|---|---|
| Copies references of nested objects | Copies nested objects too |
| Faster, less memory | Slower, more memory |
| Changes can reflect in original | Fully independent |

## Q23. Name any 3 tools for EDA and explain EDA

**EDA (Exploratory Data Analysis)** is the process of examining a dataset to understand its structure, find patterns, spot missing values and outliers, check distributions and relationships, and test assumptions **before** building a model. Introduced by John Tukey.

**Typical steps:** load data, `head()`, `info()`, `describe()`, check missing values and duplicates, handle outliers, univariate / bivariate analysis, correlation heatmap.

**3 tools for EDA**

1. **Pandas Profiling (ydata-profiling):** generates a full automatic report in one line.
2. **Sweetviz:** automatic EDA report with comparison between train and test data.
3. **D-Tale / Seaborn + Matplotlib / AutoViz:** interactive or manual visual analysis.

```
# Manual EDA
print(df.shape)
df.info()
print(df.describe())
print(df.isnull().sum())
print(df.duplicated().sum())
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()

# Automatic EDA
# pip install ydata-profiling
from ydata_profiling import ProfileReport
ProfileReport(df, title='EDA Report').to_file('report.html')
```

## Q24. What is an outlier?

An **outlier** is a data point that is **very different** from the other observations: unusually high or low compared to the rest of the data.

**Example:** Salaries of 5 employees: 30k, 32k, 35k, 33k, **5,00,000**. The last value is an outlier.

**Causes:** data-entry errors, measurement errors, or genuine rare events.

**Effects:** distorts mean and standard deviation, reduces model accuracy (especially linear regression, KNN).

**Handling:** remove them, cap them (winsorization), transform (log), or use robust methods (median, tree models).

```
sal = pd.Series([30, 32, 35, 33, 500])
print(sal.mean())     # 126  -> dragged up by the outlier
print(sal.median())   # 33   -> robust
```

## Q25. Which kind of plots are used to detect outliers? Explain with example

**1. Box plot (best):** points beyond the whiskers (outside 1.5 x IQR) are shown as individual dots = outliers.

**2. Scatter plot:** a point far away from the cluster of other points is an outlier (good for two variables).

**3. Histogram:** isolated bars far from the main distribution.

Others: violin plot, Z-score plot, distribution (KDE) plot.

```
data = pd.DataFrame({'Salary': [30, 32, 35, 33, 31, 34, 36, 500],
                     'Age':    [25, 27, 30, 28, 26, 29, 31, 28]})

sns.boxplot(y=data['Salary'])          # dot at 500 = outlier
plt.show()

plt.scatter(data['Age'], data['Salary'])   # one point far above the others
plt.show()

plt.hist(data['Salary'], bins=10)          # isolated bar near 500
plt.show()
```

## Q26. In a boxplot, what does the size and shape of the box tell us?

The box spans from **Q1 (25th percentile) to Q3 (75th percentile)**, so it holds the middle 50% of the data. The line inside is the **median (Q2)**. Whiskers extend to the lowest and highest values within 1.5 x IQR; dots beyond are outliers.

**Size (height/length) of box = IQR = Q3 - Q1:**

- Big box: data is widely spread (high variability).
- Small box: data is tightly packed (low variability, consistent).

**Shape / position of median inside the box = skewness:**

- Median in the centre, whiskers equal: **symmetric** (normal-like) distribution.
- Median closer to Q1 (bottom), longer upper whisker: **right (positive) skew**.
- Median closer to Q3 (top), longer lower whisker: **left (negative) skew**.

```
np.random.seed(0)
df = pd.DataFrame({
    'Tight':  np.random.normal(50, 2, 200),     # small box
    'Spread': np.random.normal(50, 15, 200),    # big box
    'Skewed': np.random.exponential(10, 200)    # median near bottom
})
sns.boxplot(data=df)
plt.show()
```

## Q27. What is IQR? Explain in detail

**IQR (Interquartile Range)** = **Q3 - Q1**. It measures the spread of the **middle 50%** of the data and is **not affected by outliers**, unlike range or standard deviation.

- Q1 = 25th percentile, Q2 = median (50th), Q3 = 75th percentile.

**Outlier rule:**

- Lower fence = Q1 - 1.5 x IQR
- Upper fence = Q3 + 1.5 x IQR
- Any value outside the fences is an outlier.

**Worked example:** data = 2, 4, 5, 7, 8, 9, 10, 12, 100.

Q1 = 4.5 (approx), Q3 = 10 (approx), IQR = 5.5. Upper fence = 10 + 8.25 = 18.25, so **100 is an outlier**.

```
s = pd.Series([2, 4, 5, 7, 8, 9, 10, 12, 100])

Q1 = s.quantile(0.25)
Q3 = s.quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
print(Q1, Q3, IQR, lower, upper)

outliers = s[(s < lower) | (s > upper)]
print(outliers)                      # 100

clean = s[(s >= lower) & (s <= upper)]   # remove outliers
```

# Q28. What is regression and its types?

**Regression** is a supervised learning technique used to predict a **continuous numerical value** from one or more independent variables by finding the relationship between them. Example: predicting salary from experience.

**Types of regression**

1. **Simple Linear Regression:** one input, straight line. y = mx + c.
2. **Multiple Linear Regression:** many inputs. y = b0 + b1x1 + b2x2 + ...
3. **Polynomial Regression:** curved relationship (x squared, x cubed).
4. **Ridge Regression (L2):** linear regression with a penalty to reduce overfitting.
5. **Lasso Regression (L1):** penalty that can shrink some coefficients to zero (feature selection).
6. **ElasticNet:** mix of Ridge and Lasso.
7. **Logistic Regression:** despite its name, used for classification (see Q29).
8. **Decision Tree / Random Forest / SVR:** non-linear regression models.

```
from sklearn.linear_model import LinearRegression

exp = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)    # years of experience
salary = np.array([30, 35, 40, 45, 50])           # in thousands

model = LinearRegression().fit(exp, salary)
print(model.coef_, model.intercept_)   # [5.] 25.0   -> salary = 5*exp + 25
print(model.predict([[6]]))            # [55.]
```

# Q29. Why is logistic regression called a classification algorithm?

The name says "regression" because it uses a **linear equation** (z = b0 + b1x1 + ...) internally. But the output is passed through the **sigmoid function**, which squeezes any value into a probability between 0 and 1:

**p = 1 / (1 + e^(-z))**

A threshold (usually 0.5) then converts this probability into a **class label**: p >= 0.5 gives class 1, otherwise class 0. Since the final output is a **category** (yes/no, spam/not spam, pass/fail) and not a continuous number, it is a **classification** algorithm.

```
from sklearn.linear_model import LogisticRegression

hours = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
passed = np.array([0, 0, 0, 1, 1, 1])

clf = LogisticRegression().fit(hours, passed)
print(clf.predict([[2.5], [5]]))          # [0 1]   -> class labels
print(clf.predict_proba([[2.5], [5]]))    # probabilities for each class
```

Multi-class problems are handled with softmax / one-vs-rest.

## Q30. What is the curse of dimensionality?

As the **number of features (dimensions)** increases, the amount of data needed to cover the space grows **exponentially**. In high dimensions, data becomes **sparse**, distances between points become almost the same, and models **overfit**, become slow and perform badly.

**Problems caused:**

- Data sparsity and need for much more training data
- Distance-based algorithms (KNN, K-Means) fail since all points look equally far
- Overfitting and high computation / memory cost
- Hard to visualize

**Solutions:**

- Feature selection (Q35)
- Dimensionality reduction: PCA, t-SNE, LDA
- Collecting more data, regularization

```
from sklearn.decomposition import PCA
from sklearn.datasets import load_digits

X = load_digits().data            # 64 features
print(X.shape)                    # (1797, 64)
X_small = PCA(n_components=10).fit_transform(X)
print(X_small.shape)              # (1797, 10)
```

# Q31. Difference between shape and size property

- **shape:** a tuple giving the **number of elements along each dimension** (rows, columns, ...).
- **size:** the **total number of elements** (a single integer = product of the shape).

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a.shape)   # (2, 3)  -> 2 rows, 3 columns
print(a.size)    # 6       -> 2 x 3
print(a.ndim)    # 2       -> number of dimensions

df = pd.DataFrame({'A':[1,2,3], 'B':[4,5,6]})
print(df.shape)  # (3, 2)
print(df.size)   # 6   (rows x columns)
```

| shape | size |
|---|---|
| Tuple, e.g. (2, 3) | Integer, e.g. 6 |
| Dimension-wise count | Total count |
| Needed for reshape | Used to check reshape feasibility |

Both are **properties** (no brackets), not functions.

# Q32. How will you create a transpose of a matrix?

The **transpose** swaps rows and columns: element at (i, j) moves to (j, i). A (2 x 3) matrix becomes (3 x 2).

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])

print(m.T)                    # property
print(np.transpose(m))        # function
print(m.transpose())          # method
# [[1 4]
#  [2 5]
#  [3 6]]

# pandas DataFrame
df = pd.DataFrame(m, columns=['a','b','c'])
print(df.T)

# pure Python (without NumPy)
print([list(r) for r in zip(*m.tolist())])
```

For higher dimensions use `np.transpose(arr, axes=(1,0,2))`. Note: `.T` returns a view, not a copy.

## Q33. What is KNN?

**K-Nearest Neighbours** is a simple supervised algorithm (classification and regression). It is a **lazy learner**: no training, it just stores the data. To predict a new point it:

1. Chooses K (e.g. 3 or 5).
2. Calculates the distance (usually **Euclidean**) from the new point to all training points.
3. Picks the K nearest points.
4. **Classification:** takes the majority vote. **Regression:** takes the average.

**Example:** fruit by weight and size. New fruit's 3 nearest neighbours are 2 apples and 1 orange, so it is classified as **apple**.

**Choosing K:** odd K avoids ties; small K = noisy / overfit, large K = too smooth / underfit.

**Pros:** simple, no training, works for non-linear data. **Cons:** slow on large data, sensitive to outliers, **needs feature scaling**, suffers from curse of dimensionality.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
random_state=42)

sc = StandardScaler().fit(X_tr)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(sc.transform(X_tr), y_tr)
print(knn.score(sc.transform(X_te), y_te))   # accuracy
```

## Q34. Which encoding technique creates a separate column in the dataset, and for which type of ML problem is it used?

(I read the handwriting as "creates separate column"; if your question says "single column", the answer is Label Encoding, which keeps one column.)

**One-Hot Encoding** creates a **new binary (0/1) column for every category** of a feature. The original column is replaced by these columns.

**Used in: supervised learning** problems (both **classification** and **regression**) with **nominal categorical features** (no natural order), e.g. city, colour, gender. It is essential for models that treat numbers as magnitudes: linear/logistic regression, KNN, SVM, neural networks. Tree-based models can often cope with label encoding.

```
df = pd.DataFrame({'City': ['Delhi', 'Pune', 'Delhi', 'Agra']})

print(pd.get_dummies(df, columns=['City'], dtype=int))
#    City_Agra  City_Delhi  City_Pune
# 0          0           1          0
# 1          0           0          1
# 2          0           1          0
# 3          1           0          0

# scikit-learn version
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(sparse_output=False)
print(enc.fit_transform(df[['City']]))
```

Tip: use `drop_first=True` to avoid the dummy-variable trap (multicollinearity) in linear models.

## Q35. What do you mean by feature selection?

**Feature selection** is the process of choosing the **most relevant features** (columns) for the model and discarding irrelevant or redundant ones.

**Why:** better accuracy, less overfitting, faster training, simpler and more interpretable model, helps against the curse of dimensionality.

**Methods**

1. **Filter methods:** score features statistically before modelling (correlation, chi-square, ANOVA F-test, variance threshold).
2. **Wrapper methods:** train models on feature subsets (Forward selection, Backward elimination, RFE).
3. **Embedded methods:** selection happens during training (Lasso, tree feature importance).

```
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectKBest, f_classif, RFE
from sklearn.linear_model import LogisticRegression

X, y = load_iris(return_X_y=True)

# Filter: top 2 features by ANOVA F-score
sel = SelectKBest(score_func=f_classif, k=2).fit(X, y)
print(sel.get_support())          # boolean mask of kept features
print(sel.scores_)

# Wrapper: Recursive Feature Elimination
rfe = RFE(LogisticRegression(max_iter=200), n_features_to_select=2).fit(X, y)
print(rfe.support_)

# Embedded: tree importance
from sklearn.ensemble import RandomForestClassifier
print(RandomForestClassifier(random_state=0).fit(X, y).feature_importances_)
```

Difference from dimensionality reduction (PCA): feature selection **keeps original columns**; PCA creates **new** combined features.

# Q36. What is KDE? Explain with example

**KDE (Kernel Density Estimation)** is a non-parametric way to estimate the **smooth probability density curve** of a continuous variable from sample data. It is a smoothed alternative to a histogram: instead of bars, it places a small smooth "bump" (kernel, usually Gaussian) on every data point and adds them all up.

**Key parameter: bandwidth.** Small bandwidth gives a wiggly, overfit curve; large bandwidth gives an over-smooth curve. The area under the curve is 1.

**Uses:** see distribution shape (skew, multiple peaks), compare groups, detect outliers, estimate probabilities.

**Example:** heights of students: KDE shows the most common height as the peak of the curve.

```
heights = np.random.normal(165, 8, 500)

# Seaborn
sns.kdeplot(heights, fill=True, bw_adjust=1)
plt.title('KDE of Heights'); plt.show()

# Histogram + KDE together
sns.histplot(heights, kde=True, stat='density')
plt.show()

# Pandas
pd.Series(heights).plot.kde()

# Estimate density with SciPy
from scipy.stats import gaussian_kde
kde = gaussian_kde(heights)
print(kde(165))      # density at height 165

# Compare two groups
sns.kdeplot(data=df, x='Marks', hue='Gender')
```

---

**Quick revision tip:** in exams, write the definition first, then a point-wise explanation, then a small code example, then a comparison table where the question says "difference".